

Giải pháp thu thập, xử lý và phân tích dữ liệu hành vi trong hệ thống microservices



Giáo viên hướng dẫn: PGS. TS. Trương Ninh Thuận
Sinh viên thực hiện: Lê Văn Quốc
Mã sinh viên: 21020385

Nội dung chính

01

Đặt vấn đề

02

Cơ sở lý thuyết

03

Giải pháp

04

Thực nghiệm

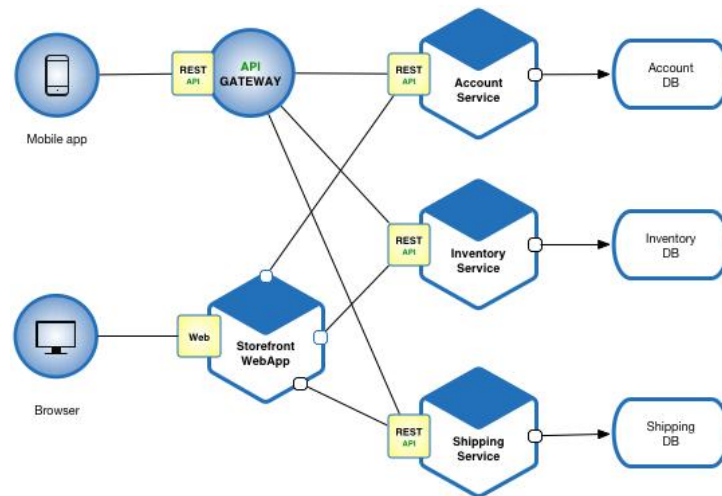
05

Kết luận và hướng phát triển

01 Đặt vấn đề

Kiến trúc microservices

- Nhiều dịch vụ nhỏ, độc lập làm việc cùng với nhau để tạo thành một hệ thống lớn
- Các dịch vụ giao tiếp với nhau thông qua các giao thức như HTTP, gRPC hoặc message broker
- Khả năng mở rộng linh hoạt, triển khai độc lập từng service



Vấn đề của hệ thống microservices

- Số lượng services lớn, phức tạp gây khó khăn trong việc hiểu kiến trúc hệ thống và giải quyết lỗi phần mềm
- Số lượng chức năng nhiều khiến việc bảo trì, tối ưu tính năng cũ và phát triển tính năng mới tốn kém chi phí, tài nguyên
- Lượng dữ liệu sinh ra rất lớn và không đồng nhất từ nhiều nguồn (service) khác nhau

→ Nhu cầu về một giải pháp hiệu năng cao để thu thập và phân tích dữ liệu, giúp giảm chi phí bảo trì hệ thống, tăng chất lượng sản phẩm cũng như trải nghiệm người dùng



02 Cơ sở lý thuyết

Khái niệm dữ liệu hành vi



Dữ liệu hành vi người dùng

- Dữ liệu thể hiện hành động và tương tác của người dùng trong môi trường số
- Dữ liệu có thể được thu thập ở cả phía client và phía server
- Đóng vai trò quan trọng trong việc duy trì và phát triển chất lượng dịch vụ



Dữ liệu hành vi hệ thống

- Dữ liệu thể hiện hành động, tương tác và phản ứng của các thành phần con cấu thành nên hệ thống khi chúng nhận đầu vào, xử lý dữ liệu và tạo ra đầu ra
- Dữ liệu được thu thập qua các kỹ thuật giám sát hệ thống
- Giúp hiểu rõ kiến trúc hệ thống, nhanh chóng sửa lỗi khi có vấn đề xảy ra

Lý thuyết nền tảng

Chuẩn OpenAPI

- Endpoint, method
- Tham số input/output
- Phương thức xác thực
- Metadata

Chuẩn OpenTelemetry

- Traces
- Metrics
- Logs
- Context propagation

➡ Các chuẩn trên cung cấp dữ liệu được chuẩn hóa đầy đủ, kèm các công cụ, thư viện hỗ trợ để tiếp cận giúp nhanh chóng cập nhật cho sản phẩm sẵn có hoặc triển khai cho sản phẩm mới

03

Giải pháp

Kiến trúc tổng quan

1 Thu thập dữ liệu

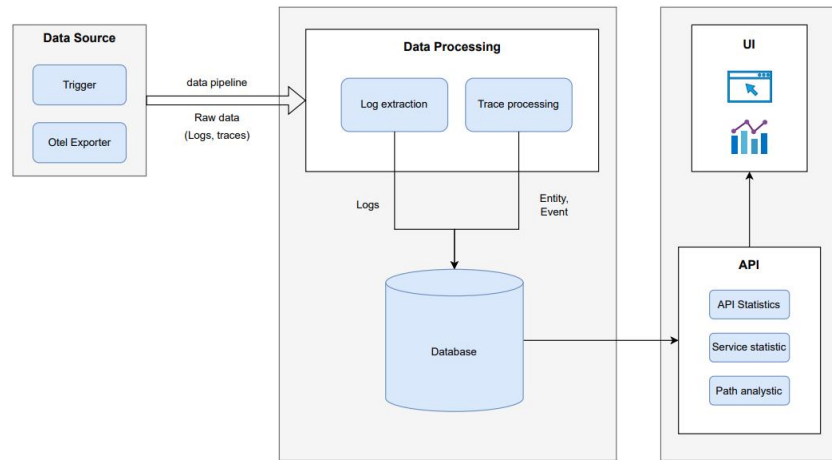
- Trigger thu thập dữ liệu request từ người dùng
- Otel Exporter thu thập dữ liệu luồng xử lý request

2 Xử lý dữ liệu

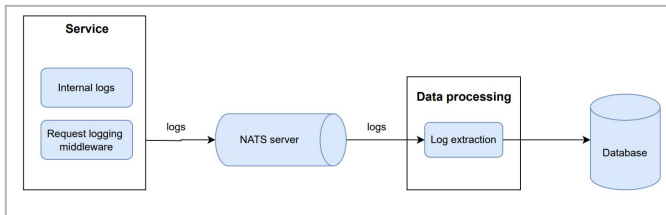
- Chuẩn hóa dữ liệu dưới dạng event
- Xây dựng luồng request bên trong hệ thống

3 Phân tích dữ liệu

- Thống kê API
- Thống kê Service
- Phân tích luồng xử lý



Thu thập dữ liệu



1

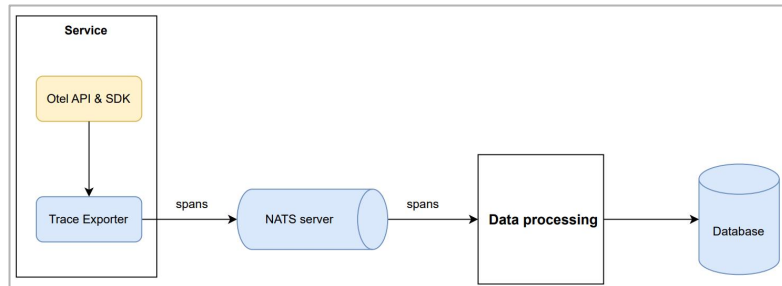
Request data

- Dữ liệu thông tin của request
- Tạo và gửi tới NATS thông qua middleware
- Phân tích hành vi người dùng sử dụng chức năng hệ thống

2

Trace data

- Dữ liệu luồng xử lý request
- Tạo ra bởi Otel API và gửi tới NATS thông qua một exporter
- Phân tích hành vi hệ thống, hiểu cách mà hệ thống hoạt động



Xử lý dữ liệu

Request log

- Log filter
- Fields filter

→ Lưu dưới dạng event phục vụ truy vấn time range

Internal log

- Thông tin tracing được thêm vào log

→ Lưu dữ liệu vào elasticsearch để truy vấn theo trace context, phục vụ debugging

Tên trường	Kiểu dữ liệu	Mô tả
timestamp	Number	Thời điểm ghi nhận sự kiện (theo chuẩn thời gian quốc tế).
service_name	String	Tên của service phát sinh log.
endpoint	String	API endpoint được gọi.
duration	Number	Thời gian xử lý request, tính bằng milliseconds.
query_params	String	Tham số truy vấn đi kèm request, dạng JSON hoặc chuỗi URL-encoded.
request_size	Number	Kích thước request gửi lên (đơn vị byte).
response_size	Number	Kích thước response trả về (đơn vị byte).
user_id	String	Mã định danh người dùng (nếu có).
user_agent	String	Thông tin trình duyệt/thiết bị của người dùng.
trace_id	String	Mã định danh trace dùng trong distributed tracing.
span_id	String	Mã định danh span tương ứng với request hiện tại.
status_code	Number	Mã trạng thái HTTP response.
error_message	String	Thông điệp lỗi (nếu có). Nếu request thành công thì để null hoặc rỗng.

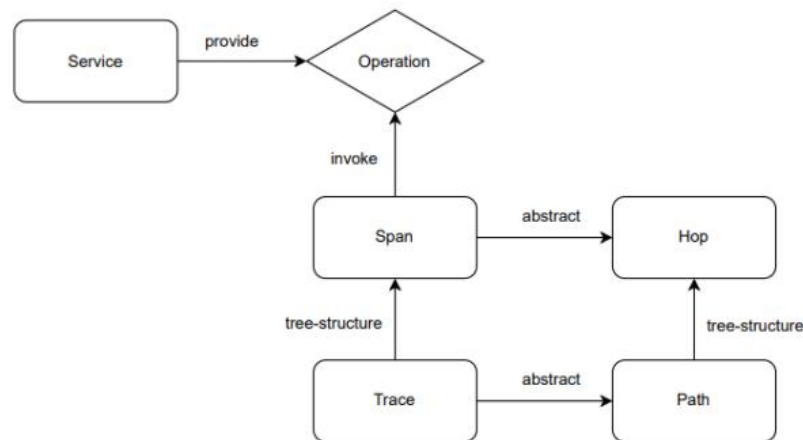
Tên trường	Kiểu dữ liệu	Mô tả
timestamp	Number	Thời điểm ghi nhận log nội bộ.
traceID	String	Mã định danh trace chứa log này.
spanID	String	Mã định danh span nơi log này được ghi nhận.
message	String	Nội dung log (thường là thông điệp lỗi, cảnh báo hoặc thông tin nghiệp vụ).

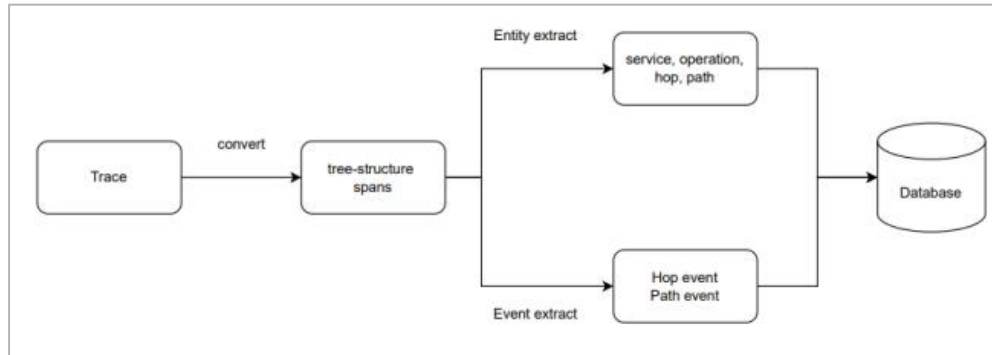
Xử lý dữ liệu

Dữ liệu tracing

- Sử dụng 2 khái niệm Hop và Path để xây dựng các luồng nghiệp vụ
- Extract 2 dạng dữ liệu:
Entity: service, operation, hop, path
Entity event: hop_event, path_event

➔ Các Entity là những đại lượng không đổi
Module xử lý sẽ thu thập các event liên quan tới các entity này để lưu vào MongoDB phục vụ cho các truy vấn time range



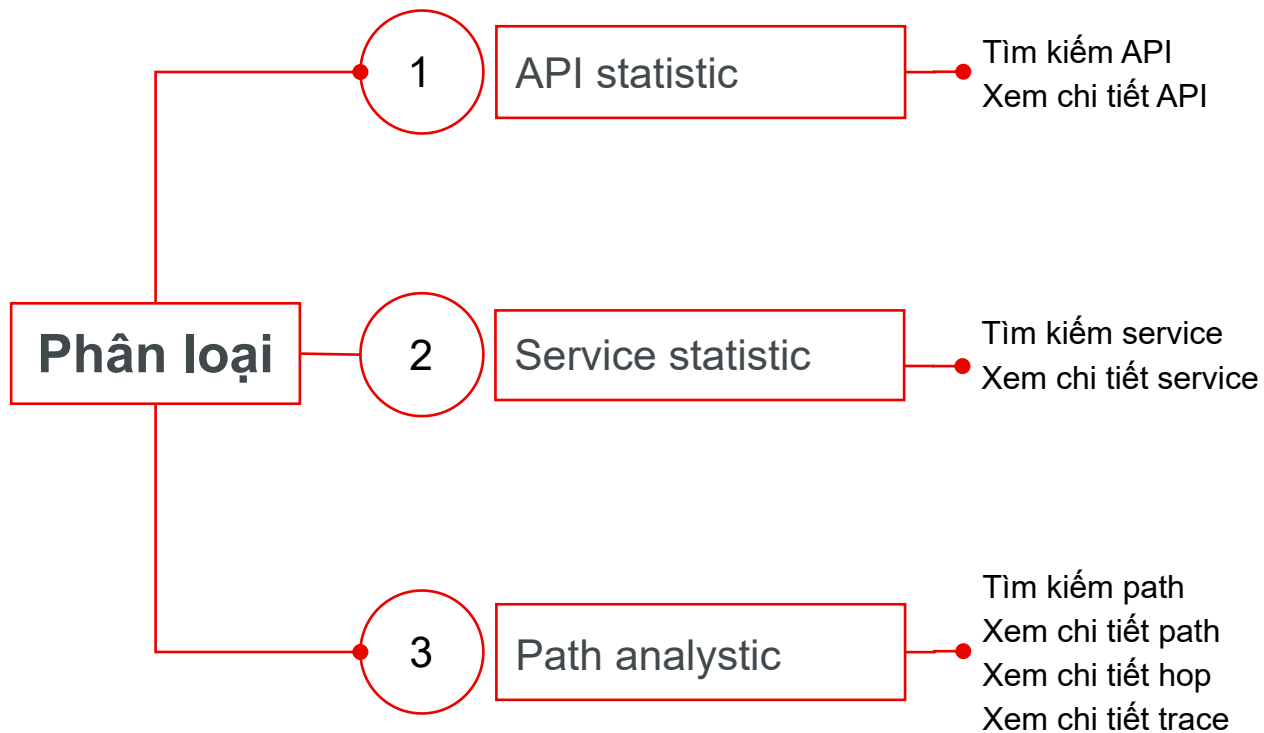


Trace processing

```
SpanHash(root, level):  
    hash = HashCode(root.serviceName) + HashCode(  
        root.operationName  
        + level x 31  
    for child in root.children:  
        hash+=SpanHash(child, level + 1)  
  
    return hash
```

Thuật toán tạo PathID

Phân tích dữ liệu



04 Thực nghiệm

Phân tích hành vi người dùng

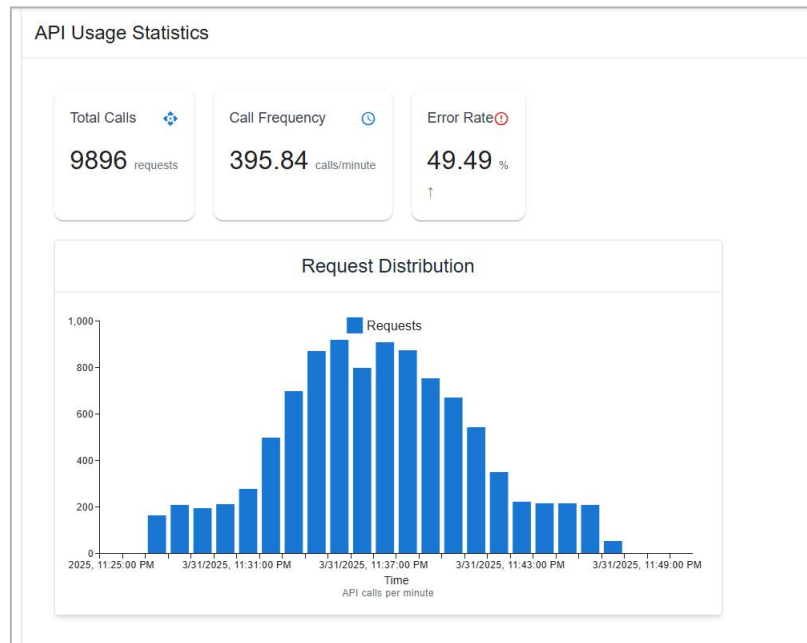
Tình huống: PM muốn hiểu khi nào người dùng gọi API thường xuyên nhất trong ngày

Phân tích:

- Thống kê số lượng request API theo khung giờ (0h–24h)
- Lọc riêng theo từng endpoint

Kết quả:

- API */checkout* thường được gọi nhiều từ 11h–13h và 20h–22h
- Đội hạ tầng auto-scale thêm instance trong khung giờ cao điểm, tiết kiệm tài nguyên còn lại



 API Latency Analysis

Find APIs that exceed a specified latency threshold

From time
2025-03-31 00:00:00

To time
2025-04-01 00:00:00

Latency Threshold (ms)
1000

Reset

Submit

APIs with latency above this value will be shown

Found 1 APIs exceeding latency threshold

Service Name	URI Path	Method	Exceed Count ↓	Average Latency
checkout-service	/checkout	POST	9897	18005 ms

Rows per page: 5 1-1 of 1 < >

Danh sách các API có độ trễ cao

View most called APIs

From time
2025-03-31 00:00:00

To time
2025-04-01 00:00:00

Reset

Submit

Top 10 called APIs

STT	Service	URI path	Method	Count ↓	Error Count	Error Rate
1	checkout-service	/checkout	POST	9897	4898	49.49%

Danh sách các API được gọi nhiều nhất

Phân tích hành vi hệ thống

Hiểu về kiến trúc hệ thống

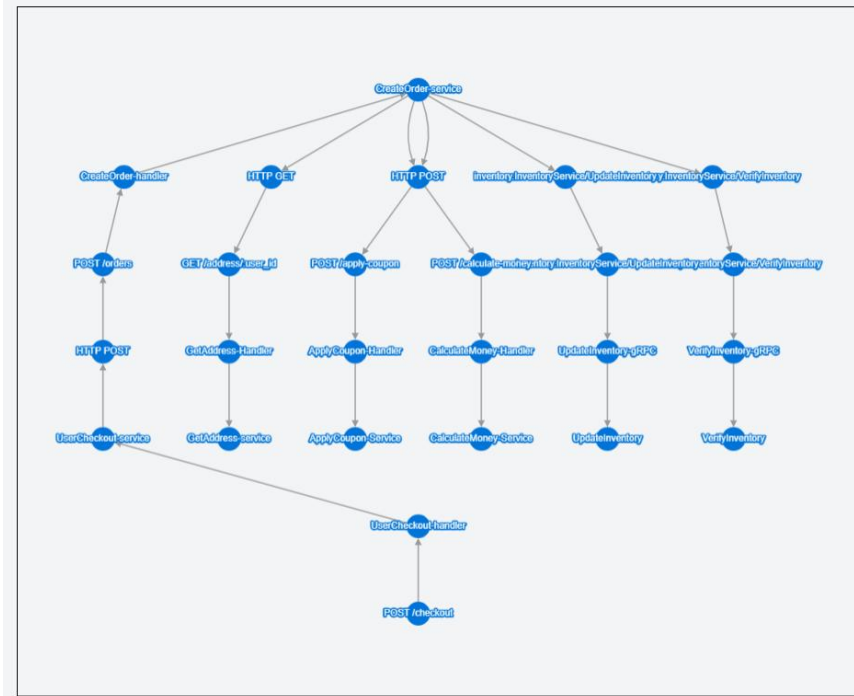
Tình huống: Developer mới A muốn tìm hiểu về luồng nghiệp vụ thanh toán của hệ thống

Phân tích:

- A thực hiện truy vấn để tìm tất cả các path nghiệp vụ có chứa **service payment**

Kết quả:

- Developer A nhanh chóng nắm được kiến trúc và thứ tự xử lý nghiệp vụ thanh toán, giảm đáng kể thời gian onboarding nhân sự mới
- Tăng độ chính xác trong hiểu nghiệp vụ, giảm lỗi phát triển



Phân tích hành vi hệ thống

Phục vụ debugging

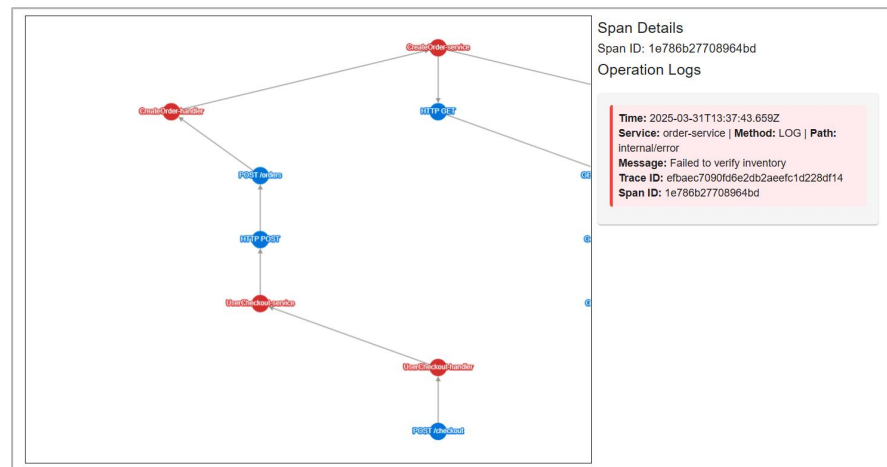
Tình huống: Tỷ lệ lỗi tăng cao trong nghiệp vụ “Tạo đơn hàng”

Phân tích:

- Developer B tìm kiếm tất cả các path đi qua operation */create-order* của **Order service**
- Xem chi tiết 1 path có lỗi trong thời gian hiện tại, B phát hiện lỗi xảy ra khi luồng xử lý gọi đến downstream **inventory service** tại operation */verify-inventory*
- Truy vấn vào trace chi tiết của path đó, B đọc được log gắn với span lỗi. Log chỉ rõ lỗi do truy vấn sai cú pháp với **database inventory**

Kết quả:

- Tiến hành sửa lỗi, test lại và deploy fix nhanh chóng
- Giảm thời gian downtime đáng kể, tăng hiệu quả vận hành và độ tin cậy hệ thống



05 Kết luận và hướng phát triển

Kết quả đạt được



- Tích hợp module thu thập dữ liệu hành vi của người dùng và hệ thống trong môi trường Microservices, đảm bảo tính chính xác và hiệu suất cao.
- Thiết kế pipeline xử lý dữ liệu để đảm bảo khả năng truyền tải nhanh chóng, mở rộng linh hoạt và giảm độ trễ trong quá trình xử lý.
- Phát triển API và giao diện phân tích thống kê trực quan những góc nhìn quan trọng trong hệ thống cho các nhà phát triển ứng dụng, cung cấp công cụ hỗ trợ giảm chi phí bảo trì, sửa lỗi, tập trung phát triển các tính năng phần mềm mang lại giá trị cao cho người dùng

Định hướng phát triển

1

Mở rộng dữ liệu thu thập

2

Ứng dụng AI/ML

3

Tích hợp hệ sinh thái giám sát



Thanks!



Do you have any questions?